

Deliverable 4: Integration and Security Implementation

Nabiha Sadaf, Nasser Hasan Padilla, Samia Zaman & Tanmay Siwach

University of the Cumberland

Advanced Operating Systems (MSCS-630-M80)

Professor Primus Vekuh

August 02, 2026

Advanced Shell: Final System Integration and Security Implementation

Abstract

This project presents the final implementation of the Advanced Shell developed throughout the previous deliverables. The objective of Deliverable 4 was to integrate all previously implemented operating system components into a unified shell while introducing advanced features including command piping, user authentication, and role-based file permissions. The completed shell provides built-in command execution, process management, CPU scheduling simulations, virtual memory management, process synchronization, command pipelines, and security mechanisms within a single interactive environment. User authentication requires valid credentials before shell access is granted, while file permissions simulate operating system access control through administrator and standard user roles. Command piping enables the output of one command to serve as the input of another, supporting chained command execution. The final implementation demonstrates how multiple operating system concepts can be integrated into a cohesive software system while maintaining modularity, usability, and security.

Keywords: operating systems, shell, command piping, authentication, file permissions, process management

Integration and Security Implementation

Introduction

Modern operating systems provide significantly more than simple command execution. They integrate process management, scheduling, memory management, synchronization, security, and command interpretation into a unified environment that allows users to efficiently interact with computer resources. Throughout the previous deliverables, each of these operating system

concepts was implemented independently within the Advanced Shell project. Deliverable 4 focused on integrating all previously developed components into a single cohesive shell while introducing security mechanisms and command piping.

The completed shell now supports foreground and background process execution, job management, CPU scheduling simulations, virtual memory management using FIFO and LRU page replacement, producer-consumer synchronization, user authentication, role-based file permissions, and command pipelines. These features collectively simulate many of the responsibilities of a traditional operating system shell while demonstrating practical implementations of operating system concepts discussed throughout the course (Silberschatz et al., 2018).

Integration Overview

The final version of the Advanced Shell integrates every major subsystem developed throughout the previous deliverables into a single command-line environment. Rather than functioning as independent demonstrations, each module is now accessible through a unified shell interface.

The shell maintains modular software architecture by separating responsibilities among multiple Python modules while allowing them to communicate through the main shell controller.

The process management subsystem continues to provide foreground and background process execution while tracking active jobs. CPU scheduling algorithms developed during Deliverable 2 remain available through dedicated scheduling commands that simulate Round Robin and Priority scheduling algorithms. Deliverable 3 components, including virtual memory management and producer-consumer synchronization, remain fully integrated and accessible through the shell's command interface.

Deliverable 4 extends this functionality by introducing authentication before shell access is granted. Once authenticated, each user operates under a defined security role that determines which operations may be performed on protected files. In addition, a command piping subsystem allows users to connect multiple commands together so that the output generated by one command becomes the input of another, closely resembling the behavior of traditional Unix shells (Silberschatz et al., 2018).

User Authentication

Before users may access any shell functionality, they must successfully authenticate using a valid username and password. During initialization, the shell loads the authentication database containing multiple user accounts. Passwords are stored using salted PBKDF2 hashing rather than plaintext values, providing significantly greater security by preventing direct disclosure of user credentials.

The authentication system distinguishes between administrator and standard user accounts. After successful login, the shell stores the authenticated user's identity and role, allowing subsequent commands to determine whether requested operations should be permitted or denied. This approach demonstrates the principle of authentication followed by authorization, where identity verification occurs before access rights are evaluated (Stallings, 2018).

Figure 1

User authentication and successful login using a standard user account.

```
C:\Users\nasse\AdvancedShell_Deliverable4>py shell.py
=====
Advanced Shell - Deliverable 4
Integration and Security Implementation
=====
Includes process management, CPU scheduling, paging, synchronization, command piping, authentication, and role-based file permissions.

User Authentication
=====
Username: student
Password:
Authentication successful. Welcome, student.
Permission level: standard

Type 'help' to view commands.
Use 'exit' to terminate the shell.

AdvancedShell:student(standard):AdvancedShell_Deliverable4$ |
```

Figure 1. Successful authentication using a standard user account before shell access is granted.

Figure 1 demonstrates the authentication mechanism implemented in the integrated shell. Before users can access any shell functionality, they must provide valid credentials consisting of a username and password. After successful authentication, the shell displays the user's permission level (standard or administrator) and grants access to the command interface. This implementation simulates secure login procedures commonly found in modern operating systems and establishes the foundation for enforcing role-based access control throughout the shell.

Command Piping Implementation

One of the primary objectives of Deliverable 4 was implementing command piping. Piping allows the output generated by one command to become the input for another command, enabling users to construct complex command sequences from simple utilities. The shell introduces a dedicated pipeline execution module responsible for parsing pipeline operators, executing commands sequentially, and forwarding output between stages.

Unlike traditional command execution, pipeline processing captures command output and forwards it directly to subsequent commands without requiring intermediate files. The implementation supports both built-in shell commands and external executables. In addition,

multiple pipeline stages may be chained together, allowing users to perform filtering, sorting, and counting operations through a single command sequence.

Figure 2

Demonstration of command piping using the `ls | grep py` pipeline.

```
AdvancedShell:student(standard):AdvancedShell_Deliverable4$ ls | grep py
__pycache__/
authentication.py
commands.py
jobs.py
memory_manager.py
permissions.py
piping.py
process_manager.py
scheduler.py
shell.py
synchronization.py
AdvancedShell:student(standard):AdvancedShell_Deliverable4$ |
```

Figure 2. Command piping between the directory listing and grep filter.

Figure 2 demonstrates the shell's command piping capability. The `ls` command generates a list of files in the current directory, and its output is passed directly to the `grep py` command, which filters the results to display only Python-related files. This implementation illustrates how the output of one command can serve as the input to another, enabling users to combine multiple commands into efficient processing pipelines.

The shell also supports multiple pipeline stages, allowing several commands to execute sequentially.

Figure 3

Multiple-command pipeline demonstrating filtering, sorting, and counting.

```
AdvancedShell:student(standard):AdvancedShell_Deliverable4$ cat test_files/application.log | grep error | sort | count
Lines      : 2
Words      : 10
Characters : 75
AdvancedShell:student(standard):AdvancedShell_Deliverable4$
```

Figure 3. Multiple-stage command pipeline using grep, sort, and count.

Figure 3 illustrates chained command execution within the shell. The output from the application log is filtered to include only error messages, sorted alphabetically, and then processed by the counting utility to summarize the resulting output. Supporting multiple pipeline stages demonstrates that the shell successfully propagates command output across several processing steps rather than limiting execution to a single pipe.

File Permission Verification

After authentication, every file operation performed by the shell is evaluated against the authenticated user's permission level. Standard users receive limited access to protected resources, whereas administrator accounts possess unrestricted access to all simulated system files.

The permission management component evaluates each requested operation before file access occurs. Read, write, and execute permissions are checked independently, allowing the shell to enforce restrictions similar to those used by modern operating systems.

Figure 4

Standard user access to a protected system configuration file.

```

AdvancedShell:student(standard):AdvancedShell_Deliverable4$ whoami
Username : student
Role     : standard
AdvancedShell:student(standard):AdvancedShell_Deliverable4$ permissions test_files/system_config.txt
Path      : test_files/system_config.txt
User      : student
Role      : standard
Permissions : read
AdvancedShell:student(standard):AdvancedShell_Deliverable4$ cat test_files/system_config.txt
Advanced Shell System Configuration
Security mode: enabled
Administrator access required for modifications.
Administrator-approved change.
AdvancedShell:student(standard):AdvancedShell_Deliverable4$ |

```

Figure 4. Read-only access to a protected configuration file by a standard user.

Figure 4 demonstrates the role-based file permission system implemented in the Advanced Shell. The authenticated user is identified as a standard user through the `whoami` command, and the `permissions` command confirms that the user has **read-only** access to the protected `system_config.txt` file. The successful execution of the `cat` command further demonstrates that standard users are permitted to view protected system files while being restricted from modifying them. This behavior simulates operating system file permission models by enforcing different access rights based on user roles, thereby improving the security of the integrated shell.

Security Mechanisms

The security subsystem extends the functionality of the shell by enforcing access control based on authenticated user roles. Rather than allowing unrestricted access to all files, the shell evaluates every read, write, and execute request against a permission database before the requested operation is performed. This design simulates the role-based access control mechanisms implemented in modern operating systems, where authorization decisions are made after successful user authentication (Stallings, 2018).

Two user roles are implemented within the shell. Administrator accounts possess unrestricted access to protected resources, while standard users receive only the permissions explicitly

granted within the file permission configuration. This separation demonstrates the principle of least privilege by ensuring users receive only the permissions necessary to perform authorized tasks.

The shell evaluates permissions before allowing file modifications, script execution, or other protected operations. Unauthorized requests generate informative access-denied messages while preventing the requested operation from executing. These mechanisms improve both the realism and security of the shell simulation.

Standard User Access Restrictions

One of the primary objectives of Deliverable 4 was demonstrating how user roles affect system security. The standard user account was intentionally limited to illustrate how operating systems prevent unauthorized users from modifying protected resources.

Figure 5

Standard user denied write and execute access to protected resources.

```
AdvancedShell:student(standard):AdvancedShell_Deliverable4$ write test_files/system_config.txt attempted-standard-user-change
security: access denied for user 'student'
security: requested operation: write
security: reason: the standard role does not have write permission for test_files/system_config.txt
AdvancedShell:student(standard):AdvancedShell_Deliverable4$ runfile test_files/admin_script.py
security: access denied for user 'student'
security: requested operation: execute
security: reason: the standard role does not have execute permission for test_files/admin_script.py
AdvancedShell:student(standard):AdvancedShell_Deliverable4$
```

Figure 5. Standard user denied write permission on a protected configuration file and execute permission on an administrator-only script.

Figure 5 demonstrates enforcement of the shell's role-based security model. When the authenticated standard user attempts to modify the protected system configuration file, the shell evaluates the user's permissions and denies the request because write access has not been

granted. Similarly, execution of the administrator-only maintenance script is rejected because the standard user lacks execute permission. These access restrictions demonstrate how authorization decisions are enforced before protected operations are performed.

Administrator Access

Administrator accounts possess full read, write, and execute privileges within the simulated file system. After successful authentication, administrator users may modify protected configuration files and execute maintenance scripts that are unavailable to standard users.

Figure 6

Administrator authentication and permission verification.

```
C:\Users\nasse\AdvancedShell_Deliverable4>py shell.py
=====
Advanced Shell - Deliverable 4
Integration and Security Implementation
=====
Includes process management, CPU scheduling, paging, synchronization, command piping, authentication, and role-based file permissions.

User Authentication
=====
Username: admin
Password:
Authentication successful. Welcome, admin.
Permission level: admin

Type 'help' to view commands.
Use 'exit' to terminate the shell.

AdvancedShell:admin(admin):AdvancedShell_Deliverable4$ |
```

Figure 6. Administrator account displaying full access permissions.

Figure 6 illustrates successful authentication using an administrator account. The `whoami` command identifies the authenticated user, while the `permissions` command confirms that administrator accounts possess read, write, and execute privileges for protected system resources. Unlike standard users, administrators are not restricted when accessing protected files because their role includes complete authorization within the simulated security model.

Figure 7

Administrator modification of a protected configuration file.

```
AdvancedShell:admin(admin):AdvancedShell_Deliverable4$ whoami
Username : admin
Role     : admin
AdvancedShell:admin(admin):AdvancedShell_Deliverable4$ permissions test_files/system_config.txt
Path      : test_files/system_config.txt
User      : admin
Role      : admin
Permissions : read, write, execute
AdvancedShell:admin(admin):AdvancedShell_Deliverable4$ permissions test_files/admin_script.py
Path      : test_files/admin_script.py
User      : admin
Role      : admin
Permissions : read, write, execute
AdvancedShell:admin(admin):AdvancedShell_Deliverable4$ |
```

Figure 7. Administrator successfully updating a protected system configuration file.

Figure 7 demonstrates successful modification of a protected system file by an administrator. The shell verifies that the authenticated user possesses write permission before allowing the update.

After the operation completes, the contents of the configuration file confirm that the new information was successfully appended. This behavior illustrates how authorization enables privileged users to perform administrative maintenance tasks while preventing unauthorized modifications by standard users.

Figure 8

Administrator execution of a protected maintenance script.

```
AdvancedShell:admin(admin):AdvancedShell_Deliverable4$ write test_files/system_config.txt Administrator-approved change.
write: appended text to test_files/system_config.txt
AdvancedShell:admin(admin):AdvancedShell_Deliverable4$ cat test_files/system_config.txt
Advanced Shell System Configuration
Security mode: enabled
Administrator access required for modifications.
Administrator-approved change.
Administrator-approved change.
AdvancedShell:admin(admin):AdvancedShell_Deliverable4$ runfile test_files/admin_script.py
Administrator script executed successfully.
Protected maintenance operation completed.
runfile: test_files\admin_script.py finished with exit code 0
AdvancedShell:admin(admin):AdvancedShell_Deliverable4$ |
```

Figure 8. Successful execution of an administrator-only maintenance script.

Figure 8 demonstrates execution of a protected Python script by an authenticated administrator. Because the administrator possesses execute permission, the shell allows the script to run successfully and reports its completion. In contrast, the same operation performed by a standard user is denied. This distinction illustrates how execute permissions contribute to overall operating system security by preventing unauthorized execution of privileged programs.

Challenges and Improvements

Integrating the features developed throughout the previous deliverables presented several implementation challenges. One of the primary challenges involved combining independently developed modules while preserving their existing functionality. Process management, scheduling, memory management, and synchronization were originally implemented as separate components and required careful integration into a unified shell without introducing compatibility issues.

Another challenge involved implementing command pipelines that supported both built-in shell commands and external executable programs. Unlike traditional command execution, pipeline operations required capturing command output and forwarding it as input to subsequent commands. Supporting multiple pipeline stages required careful management of command execution order and intermediate data.

The security subsystem also introduced several implementation considerations. Authentication required secure password storage using salted password hashing rather than plaintext credentials. File permission enforcement required authorization checks before read, write, and execute

operations could proceed. Testing confirmed that standard users were appropriately restricted while administrators retained unrestricted access to protected resources.

Although the completed shell satisfies the objectives of this project, several future enhancements could further improve its capabilities. Examples include support for user account creation, encrypted password databases, group-based permissions, command history, shell scripting, environment variables, input and output redirection, advanced scheduling algorithms, and additional memory management techniques. Implementing these features would further increase the realism of the shell while expanding its educational value.

Conclusion

Deliverable 4 successfully integrated all operating system components developed throughout the previous assignments into a unified Advanced Shell. The final implementation combines process management, job control, CPU scheduling, virtual memory management, process synchronization, command piping, user authentication, and role-based file permissions within a single interactive environment.

The shell demonstrates practical implementations of several fundamental operating system concepts. Authentication and authorization provide secure access control through administrator and standard user roles, while file permission enforcement simulates real-world operating system security. Command piping extends shell functionality by allowing multiple commands to communicate through sequential processing pipelines. Together, these features create a comprehensive shell simulation that demonstrates the interaction between operating system components while reinforcing the theoretical concepts studied throughout the course.

The completed project illustrates how modular software design can be used to integrate multiple operating system subsystems into a cohesive application. Through the combination of security mechanisms, process management, scheduling, memory management, synchronization, and command interpretation, the Advanced Shell provides a practical demonstration of core operating system principles and their application within modern computing environments.

References

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson.